

COSC 00130: SYSTEM ANALYSIS AND DESIGN

INTRODUCTION

Software: Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

As software is a collection of programs and data files that are designed to perform some operations and on the other hand, program is a set of instructions that perform only a specific task that it is made for.

An information system is integrated and co-ordinate network of components, which combine together to convert data into information.

A system is a set of related components that produces specific results.

TYPES OF SOFTWARE PRODUCTS

- Generic products These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to software buy them. Examples of this type of product include software for PCs such as databases, word processors, drawing packages, and project-management tools.
- Customized (or bespoke) products These are systems that are commissioned by a particular customer. A software contractor develops the software especially for that customer. Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

An important difference between these types of software is that, in generic products, the organization that develops the software controls the software specification. For custom products, the specification is usually developed and controlled by the organization that is buying the software. The software developers must work to that specification.

Elements of a System

- ❖ **Architecture** - This is the conceptual model that defines the structure, behavior and more views of a system. We can use flowcharts to represent and illustrate the architecture.
- ❖ **Modules** - These are components that handle one specific task in a system. A combination of the modules makes up the system.
- ❖ **Components** - This provides a particular function or group of related functions. They are made up of modules.
- ❖ **Interfaces** - This is the shared boundary across which the components of the system exchange information and relate.
- ❖ **Data** - This is the management of the information and data flow. Components of information systems

An information system is essentially made up of five components: hardware, software, database, network and people. These five components integrate to perform input, process, output, feedback and control.

Database consists of data organized in the required structure. Network consists of hubs, communication media and network devices.

Information processing consists of input; data process, data storage, output and control. During input stage data instructions are fed to the systems which during process stage are worked upon by software programs and other queries. During output stage, data is presented in structured format and reports.

- **Hardware** consists of everything in the physical layer of the information system. Hardware consists of input/output device, processor, operating system and media devices. For example, hardware can include servers, workstations, networks, telecommunications equipment, fiber-optic cables, mobile devices, scanners, digital capture devices, and other technology-based infrastructure.
- **Software** refers to the programs that control the hardware and produce the desired information or results. Software consists of system software and application software. Software consists of various programs and procedures.
- **Data** is the raw material that an information system transforms into useful information. An information system can store data in various locations, called tables.
- **Processes** describe the tasks and business functions that users, managers, and IT staff members perform to achieve specific results.
- **People** who have an interest in an information system are called **stakeholders**. Stakeholders include the management group responsible for the system, the **users** (sometimes called **end users**) inside and outside the company who will interact with the system, and IT staff members, such as systems analysts, programmers, and network administrators who develop and support the system.

CHARACTERISTICS OF A GOOD SYSTEM

1. **Organization:**

It implies structure and order. It is the arrangement of components that helps to achieve objectives.

2. **Interaction:**

It refers to the manner in which each component functions with other components of the system.

3. **Interdependence:**

It means that parts of the organization or computer system depend on one another. They are coordinated and linked together according to a plan. One subsystem depends on the output of another subsystem for proper functioning.

4. **Integration:** It refers to the holism of systems. It is concerned with how a system is tied together.

5. **Central Objective:**

A system should have a central objective. Objectives may be real or stated. Although a stated objective may be the real objective, it is not uncommon for an organization to state one objective and operate to achieve another. The important point is that users must know the central objective of a computer application early in the analysis for a successful design and conversion.

SYSTEM DEVELOPMENT METHODOLOGIES

A **system** development **methodology** refers to the framework that is used to structure, plan, and control the process of developing an information **system**.

Agile development methodology

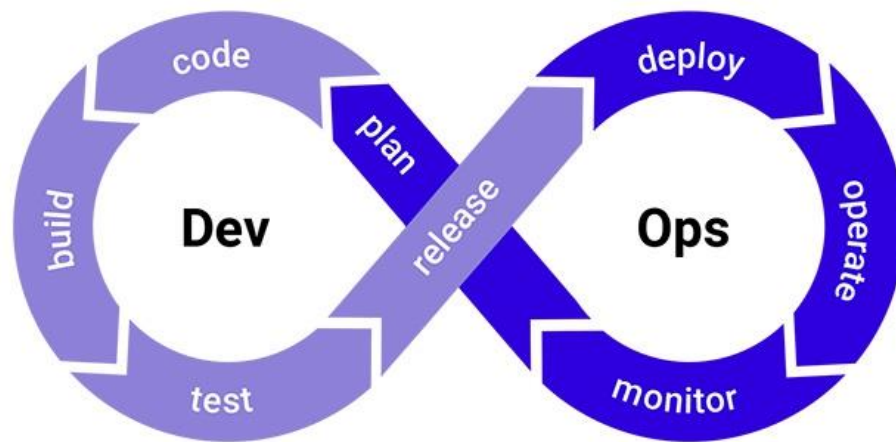
Teams use the agile development methodology to minimize risk (such as bugs, cost overruns, and changing requirements) when adding new functionality. In all agile methods, teams develop the software in iterations that contain mini-increments of the new functionality. There are many different forms of the agile development method, including scrum, crystal, extreme programming (XP), and feature-driven development (FDD).

Pros: The primary benefit of agile software development is that it allows software to be released in iterations. Iterative releases improve efficiency by allowing teams to find and fix defects and align expectation early on. They also allow users to realize software benefits earlier, with frequent incremental improvements.

Cons: Agile development methods rely on real-time communication, so new users often lack the documentation they need to get up to speed. They require a huge time commitment from users and are labor intensive because developers must fully complete each feature within each iteration for user approval.

DevOps deployment methodology

DevOps is not just a development methodology but also a set of practices that supports an organizational culture. DevOps deployment centers on organizational change that enhances collaboration between the departments responsible for different segments of the development life cycle, such as development, quality assurance, and operations.



Pros: DevOps is focused on improving time to market, lowering the failure rate of new releases, shortening the lead time between fixes, and minimizing disruption while maximizing reliability. To achieve this, DevOps organizations aim to automate continuous deployment to ensure everything happens smoothly and reliably. Companies that use DevOps methods benefit by significantly reducing time to market and improving customer satisfaction, product quality, and employee productivity and efficiency.

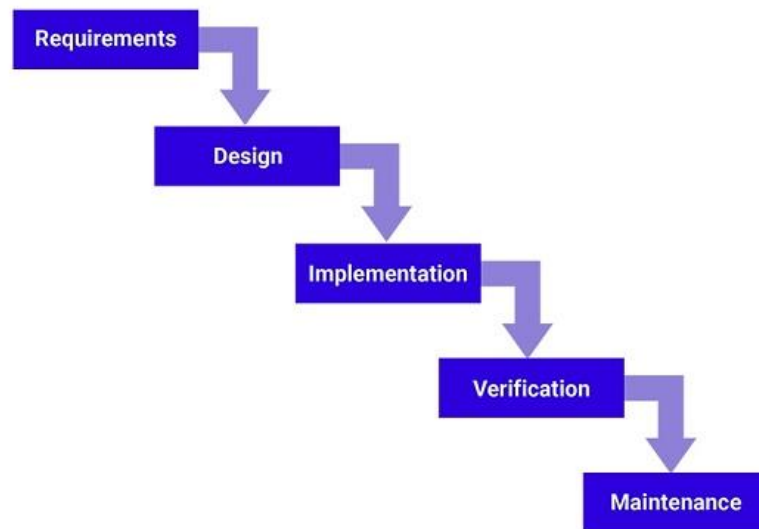
Cons: Even in light of its benefits, there are a few drawbacks to DevOps:

- Some customers don't want continuous updates to their systems.
- Some industries have regulations that require extensive testing before a project can move to the operations phase.
- If different departments use different environments, undetected issues can slip into production.

- Some quality attributes require human interaction, which slows down the delivery pipeline.

Waterfall development method

Many consider the waterfall method to be the most traditional software development method. The waterfall method is a rigid linear model that consists of sequential phases (requirements, design, implementation, verification, maintenance) focusing on distinct goals. Each phase must be 100% complete before the next phase can start. There's usually no process for going back to modify the project or direction.



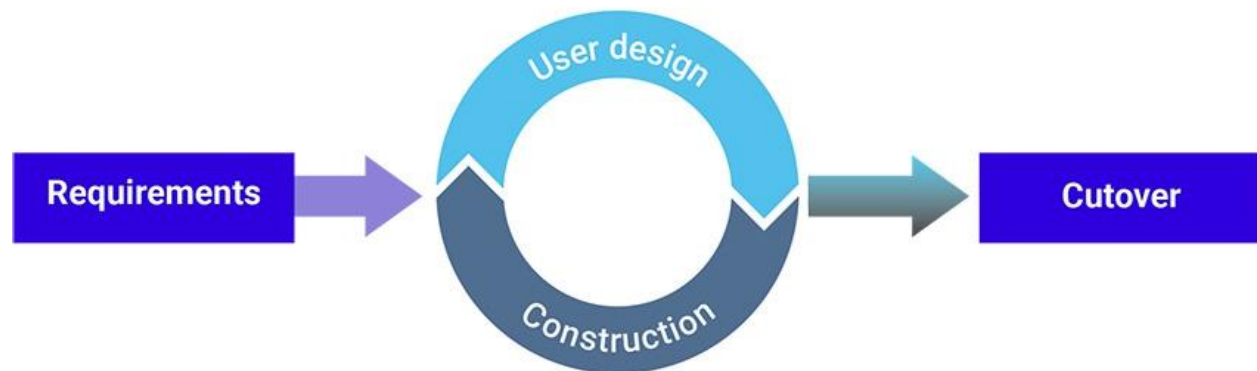
Pros: The linear nature of the waterfall development method makes it easy to understand and manage. Projects with clear objectives and stable requirements can best use the waterfall method. Less experienced project managers and project teams, as well as teams whose composition changes frequently, may benefit the most from using the waterfall development methodology.

Cons: The waterfall development method is often slow and costly due to its rigid structure and tight controls. These drawbacks can lead waterfall method users to explore other software development methodologies.

Rapid application development

Rapid application development (RAD) is a condensed development process that produces a high-quality system with low investment costs. Scott Stiner, CEO and president of UM Technologies, said in Forbes, “This RAD process allows our developers to quickly adjust to shifting requirements in a fast-paced and constantly changing market.” The ability to quickly adjust is what allows such a low investment cost.

The rapid application development method contains four phases: requirements planning, user design, construction, and cutover. The user design and construction phases repeat until the user confirms that the product meets all requirements.



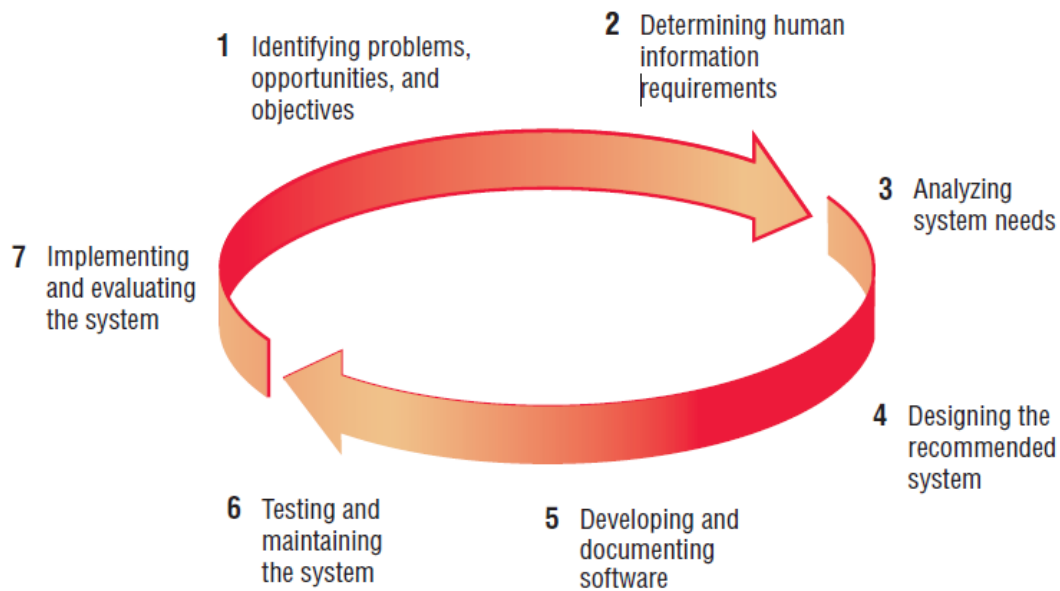
Pros: Rapid application development is most effective for projects with a well-defined business objective and a clearly defined user group, but which are not computationally complex. RAD is especially useful for small to medium projects that are time sensitive.

Cons: Rapid application development requires a stable team composition with highly skilled developers and users who are deeply knowledgeable about the application area. Deep knowledge is essential in a condensed development timeline that requires approval after each construction phase. Organizations that don't meet these requirements are unlikely to benefit from RAD.

Which software development methodology should I use?

These four software development methodologies are the most pervasive in software development. Each one has its own strengths and weaknesses and works effectively in different situations. When choosing your development methodology, think about combining the elements of each method that work best for your team and your current project. In this way, you can create a hybrid development methodology that'll get you to production securely and efficiently.

SYSTEM DEVELOPMENT LIFECYCLE



Identifying Problems, Opportunities, and Objectives

In this first phase of the systems development life cycle, the analyst is concerned with correctly identifying problems, opportunities, and objectives. This stage is critical to the success of the rest of the project, because no one wants to waste subsequent time addressing the wrong problem.

It focuses on the scope of the project of the project.

The outputs of planning phase include: project plans, schedules, cost estimation and procurement requirements.

Determining Human Information Requirements

The next phase the analyst enters is that of determining the human needs of the users involved, using a variety of tools to understand how users interact in the work context with their current information systems. The analyst will use interactive methods such as interviewing, sampling and investigating hard data, and questionnaires, along with unobtrusive methods, such as observing decision makers' behavior and their office environments, and all-encompassing methods, such as prototyping.

The IT team gathers information from all the stakeholders. The output of this phase in waterfall model is a document with the list of all requirement. Agile may produce a task to be performed.

Analyzing System Needs

The next phase that the systems analyst undertakes involves analyzing system needs. Again, special tools and techniques help the analyst make requirement determinations. Tools such as data flow diagrams (DFD) to chart the input, processes, and output of the business's functions, or activity diagrams or sequence diagrams to show the sequence of events, illustrate systems in a structured, graphical form. From data flow, sequence, or other diagrams, a data dictionary is developed that lists all the data items used in the system, as well as their specifications.

Once the requirements are understood, the design process takes place. Output include design document that list the patterns and components selected and the start of the development.

Designing the Recommended System

In the design phase of the SDLC, the systems analyst uses the information collected earlier to accomplish the logical design of the information system. The analyst designs procedures for users to help them accurately enter data so that data going into the information system are correct. In addition, the analyst provides for users to complete effective input to the information system by using techniques of good form and Web page or screen design.

Developing and Documenting Software

In the fifth phase of the SDLC, the analyst works with programmers to develop any original software that is needed. During this phase the analyst works with users to develop effective documentation for software, including procedure manuals, online help, and Web sites featuring Frequently Asked Questions (FAQs), on Read Me files shipped with new software. Because users are involved from the beginning, phase documentation should address the questions they have raised and solved jointly with the analyst. Documentation tells users how to use software and what to do if software problems occur.

The software is developed in this phase. The output is a testable functional software.

Testing and Maintaining the System

Before the information system can be used, it must be tested. It is much less costly to catch problems before the system is signed over to users. Some of the testing is completed by programmers alone, some of it by systems analysts in conjunction with programmers. A series of tests to pinpoint problems is run first with sample data and eventually with actual data from the current system. Often test plans are created early in the SDLC and are refined as the project progresses.

Maintenance of the system and its documentation begins in this phase and is carried out routinely throughout the life of the information system. Much of the programmer's routine work consists of maintenance, and businesses spend a great deal of money on maintenance. Some maintenance, such as program updates, can be done automatically via a vendor site on the Web. Many of the systematic procedures the analyst employs throughout the SDLC can help ensure that maintenance is kept to a minimum.

Implementing and Evaluating the System

In this last phase of systems development, the analyst helps implement the information system. This phase involves training users to handle the system. Vendors do some training, but oversight of training is the responsibility of the systems analyst. In addition, the analyst needs to plan for a smooth conversion from the old system to the new one. This process includes converting files from old formats to new ones, or building a database, installing equipment, and bringing the new system into production.